

Week 2: Machine Learning Fundamentals

Video Transcripts

Video 2.1: Machine Learning Approaches

In Machine learning, we have got supervised learning with its categories of applications such as regression and classification. We have got reinforcement learning with its own applications in navigation of robots, in developing intelligent games, and so on. And also, we have got unsupervised learning. So, the main item that segregates those three approaches from each other is the way that we train our machines over there. For instance, in supervised learning, the learning, the training would be basically, grammatically different from reinforcement learning and from unsupervised learning. And the applications and behaviour of supervised learning systems is different from reinforcement learning systems is different from unsupervised learning modules. And in the field of unsupervised learning, we have got some important categories, subcategories such as computing the correlation between the features or going for a kind of clustering of our data, and something like that.

Video 2.2: Supervised Learning: Definition and Components

Let's talk about supervised learning and its definition. In supervised learning, we have got the type of machine learning where the algorithm learns from a labelled data. Data is known. We have got a subset of known data with output labels known and declared for all data sample. It uses input output pairs where the input data features or our independent input variable is associated with corresponding output. Our labels our dependent variable or variables for our class labels. So, to learn a mapping function, we use that pair of input output data independent dependent variables. Using a rather proper set of those pairs we can train our machine learning models, and these models. After a while we learn how to do that mapping from inputs to outputs. The goal is to predict output for new unseen input, and for new unseen data based on the learned relationships that we have learned them from our training data.

So, after training a machine using labelled data, a supervised learning model would be able to compute to predict the output for some unseen test validation, and practical data samples. This is the definition of supervised learning. We learn that what is the relation between inputs and outputs and then after learning that we can compute and predict the output for unknown data samples. If we want to know about the components typical in supervised learning, we have got input data. They are features that can help us to categorise or predict the output. They are variables used to make predictions. For instance, if we wanna recognise a face, the pixels on that images are our features. Of course that we may go for extraction of some features from those pixels and so on. Again in text analytics, supervised learning, text analytics, for instance in sentiment analysis over features or text words and tokens. And in some cases we have got structured data, structured data with some columns as features, and input independent variables that will help us to classify the samples into a few classes or to guess the output for some new data sample.

So, in a structured data, for instance, if our data is some numbers or characters or categories in an excel sheet, okay, those columns are our features or variables. Output data are labels or output of a

function that we are going to estimate known results that the model aims to predict. For instance, categories, classes output numerical values as our dependent variable. They are our output features. So, in the case of classification, those output data are class labels or somehow they represent the classes. In the case of regression, output data is output of the function that our machine learning model is going to estimate using some regression techniques. In the field of machine learning, training process actually is another important component. In supervised learning, the algorithm adjusts itself, and its parameters using a labelled data set. We call it training data set to minimise the difference between its predictions and the true labels, and true outputs of that data samples. This is the meaning of training.

Video 2.3: Reinforcement Learning: Definition, Components and Process

If we want to have a definition for Reinforcement Learning, my friends, Reinforcement learning is a type of machine learning where an agent learns to make decisions by interacting with an environment. Our agent does a job in that environment and gets feedback. Some feedback, directly, indirectly, automatically or manually; anyway, we have got some feedback for that agent. That agent takes actions, observes the results, meaning feedback from the environment, and receives feedback in the form of rewards or penalties. We can say positive reward, negative reward, or something like this or we can call them rewards, fines, and penalties. Over time, our reinforcement learning agent learns a policy that maximises cumulative rewards.

So, your agent does something, observes the environment, it would be provided with a kind of reward or penalty, a positive or negative score and it will try to change its policy. Policy means the way that it does the job to maximise toward maximisation of average or cumulative reward, or in some cases instant rewards can be used as a metric as well. So, this is the way that your agent can go for reinforcement learning. Positive, negative, reward, penalty, changing the behaviour and policy to maximise the reward.

There are different components inside the reinforcement learning model. Agent is the learner or decision-maker, for instance a robot, a game bot, an automatic player in a game or an automatic contributor in something. For example, it could be a robot, and that robot could be a hardware robot, could be just a software robot. So, the agent is your learner which is going to do something and while doing that learn how to do that optimally. Environment is where the agent interacts with, what the agents interact with and where the agent carries out its tasks. For instance, the environment could be a room that your agent is going to clean, could be a chessboard, could be a maze if you are talking about games and something like that. So, environment is where your agent is going to do the job in that or the system that your agent is going to interact with. Action is a set of possible moves and a set of possible things that your agent can do, and different moves, different actions can be selected. One of them can be selected by your agent. Again, we have got a state. A state is the current situation or condition in the environment, the current state and condition of your environment. For instance, about a room, is it clean? Is it dirty? Where inside that room is dirty? Where inside that room? What percentage of that? Which part of that is clean or dirty, or something like that, Reward is a feedback given to your agent based on the actions and based on the state of the environment. And, it could be positive, could be negative. Policy is the strategy that your agent has taken to do their job. A sequence of actions can be considered as a kind of policy, usually a history of actions, a sequence of actions is our policy. So, policy is the strategy that your agent has opted for to do its task or tasks. Okay, process of Reinforcement Learning. Actually, the most

important part of this process is how your reinforcement learning agent learns. The agent observes the state of the environment based on a policy, could be predefined policy, could be a dynamic policy. Usually, it is a dynamic policy. Usually, at least, we have got some different scenarios and our agent would be able to select one of them. It takes an action based on that policy, fixed, dynamic or elected. Whatever, it takes an action and then the environment responds, transitioning to a new state because our agent has done something. So, the environment responds and we can see a kind of transitioning to a new state, and of course, after that there will be a kind of reward, either positive or negative, either big or small. The agent updates its policy to improve future reward. This is the last step. Your agent is going to update its policy to improve the rewards it gets.

Video 2.4: Unsupervised Learning: Definition and Components

Unsupervised learning is a type of machine learning where the algorithm learns patterns, and structures in data without any label. So, our data set hasn't got any output column or output label. And this data is a pure data, different variables, different columns, many rows. But there is no output, there is no clear label. The data consists only of input features and variables. And the algorithm, unsupervised learning algorithm attempts to discover hidden patterns, knowledge, facts and inside from that data, inside that data. So, inside that data patterns inside your data, it could be done via grouping, and clustering or going for a kind of changing the representations without any kind of explicit guidance by labelled and clear art. Unsupervised learning has got its own applications. Unsupervised learning is applicable in clustering, hierarchical clustering, dimensionality reduction in your data, anomaly detection and computing the correlation, and going for a kind of correlation analysis. So, in the case of unsupervised learning, what are the components, input data or features? They are raw unlabelled data. For instance customer demographic pixel values of an image or something like this. So, in case of structured data customer demographics, in the case of unstructured data, pixel values of an image or different acoustic samples in a voice or a piece of music. OK. Patterns or groups are structures or insights discovered by the algorithm. Unsupervised learning algorithm. For instance, clusters of similar customers, segmentation of an image into some regions that shows for instance a particular object or different objects or segmentation of image into objects and background evaluation of unsupervised learning, which is another important component in our unsupervised learning algorithm, usually involves qualitative assessment. Usually it is qualitative. However, in some cases we have got domain specific metrics. Since there are no explicit labels, we can go for a kind of a mild metrics or a kind of qualitative analysis to evaluate our unsupervised learning algorithm.

Video 2.5: Unsupervised Learning – Correlation Analysis Demo

Okay everybody, welcome to this practice session regarding unsupervised learning and correlation analysis. Okay, this is the things that we are going to do. We have got a Python code and a dataset. We are going to do correlation analysis on that dataset using our Python code. And in this dataset called celebs2_corr.xlsx, this is an Excel file actually that you can see that here in this Excel file. I'm going to show you how different things can be done regarding the correlation analysis and what is the Python code over here, cluster_corr1.ipynb is our Python code, and inside that data file celebs2_corr.xlsx we have got different columns, height, weight, etc. I will describe all of them to you and our goal is to visualise the correlation between those variables or columns that you can see

in this data file using this Python code and the environment, the platform that we are going to run our Python codes inside that is Google.colab. To do that, to use Google.colab, I believe that you are more or less familiar with that. We need a drive in our Google Drive. Whatever you have got codes, data, etc. If I were you, I copied them in the codes and the data files related to the examples that I'm going to cover. You will have them and you should copy all of them to a folder in your Google Drive. In my case, if I show you my directories in that in my Google Drive I have got emeritus 2025 module two, three, and here you are.

All files are here. We have got ipynb file which are Python notebooks files and Google.colab can run them for me. First time when you double-click, when you copy a Python notebook here just like this cluster 'cluster_corr1'.ipynb, when you double-click on that you should tell your Google Drive that okay, the app that is going to run this notebook is Google.colab. But after that, the app would be a default setting and whenever you click on 'ipynb', you will have it opened in Google.colab, just like the things that I have done here. And here you are. This is cluster_corr1.ipynb, the notebook that I'm going to review now. This is our examples. Okay, inside this Google.colab, the user interface is a kind of notebook. In this notebook, you will have some different cells. Some cells contain comments and codes like the one that you can see here and here. Some cells contain Python instructions and codes and okay, you can see them here. And then when we run that notebook, we will see the results over here. So, what these notebook is all about okay, we are going to show you a correlation and correlation analysis over here. What is my data file? You can see the name of that data file here, celebs2_corr.xlsx. Actually, different file format can be opened inside Python. One of them is standard Excel file. And if we want to see the contents of that file over here in my Google Drive, I just click on this 'correlation', 'celebs2_corr.xlsx', celebs2correlation Excel file.

Here you are. Okay so, over here in this file we have got a few columns, variables, features, whatever you want to call. Actually, this is data of some celebrities. Height in column A first column then weight, then sex. Sex 1 means female, 2 means male. Then we have got weight in British pounds over here, first column is height in centimeter, second column weight in kilogram. This column, column D is weight in British pounds. And then we have got another column I added here to show you how noise or random numbers added to your data will affect the correlation. This one is weight plus some noise. You can see that for instance the weight of this celebrity is 114.6 lbs and after adding some noise now it is 115. This one was 119.59, now it is 119.98 and so on. So, this column is actually this column multiplied by something, a coefficient, something a bit larger than two that converts kilogram to weight. This column and this column are linearly related, linearly correlated. And this column okay, same numbers, but affected by some noise. And we will see the results of adding some noise to the correlation between different variables. What about this last column in this data file?

Pure random numbers. Pure random numbers between zero and one, they haven't got anything to do with any other variables here. And this way, if we manage to compute the correlation between this one and other columns, we will realise what is the meaning of correlation coefficient between different variables in our dataset, which is an important part thing to do. This is an important thing to do in data analytics, in machine learning and so on. So, let's get back to our code. As I mentioned earlier, we can edit our Python programs here. If I want to add a cell, if this cell is a text, a common cell, I will click on that. If I want to add a Python instruction cell, code cell, I will click on that. If I want to run my notebook, you know about that, I will click on this 'Runtime'. I want to change the runtime type to CPU because I don't want to waste my GPU facilities here. This program is a very simple program. I don't want to use GPU here, so I changed that to CPU and save it. What did I do? In

Runtime's menu, I changed the Runtime type from GPU to CPU because of okay, there's the GPU services are a bit limited here in Colab. I will use them later when we want to run deep learning etc. But here that's okay. So, CPU is okay. High RAM, I can remove that as well. Because these programs are the kind of simple, we don't need high amount of RAM to run this example. And I remove that as well. Okay, now we can browse this notebook.

Over here, you can see some comments about the data file different columns. I described that this is the way that you can open your data files in Google.colab. Okay, firstly in this cell we have our libraries, necessary libraries imported pandas, NumPy, and Google.colab because I need this library and its Drive class to open my data files inside Google.colab, and I mount my drive here. Sometimes when you run that, you have to approve this, approve that, because Google warns you that a program is going to connect with your Google Drive and open your files and this is the path. Have a look at this path whenever you use that content/ drive / My Drive and where in My drive we can find those data file, I showed that you emeritus/ 2025/ Module2-3, sorry it should be Module2-3 which is the name of that folder in my Google.colab and this is the file name celebs2_corr.xlsx. So, we would be able to open that. Then we know that the first few rows of dataset, I would like to show them to you. So, I use this print and then using `df.head` you would be able to see the first five rows of that. Reading from excel, I use pandas `read_excel` function. The only parameters is the name of file and half of that file and the results would be a pandas dataframe I call it `df`. `DF` is the name of my variable. Type of that variable is a pandas dataframe contents of this file would be. This file would be opened and the contents of that would be copied into my `df` pandas dataframe variable and then we can see the contents. Then inside a data file we may have some numerical variables or columns and categorical one. Actually correlation analysis only can be done on numerical variables. This is not the case here. All my variables are numerical here. But just in case if you want to go for a correlation analysis that column, that variable should be numerical. If I want to get rid of non-numerical variable, okay. You can see this instruction here. Just include numbers.

Just include numbers and I copy numerical columns of `df` to `numeric_df`. This is not the case here. Actually all the columns would be copied and then I use these function `numeric_df.corr()` to compute the correlation matrix `numeric_df.corr()` correlation. Numeric correlation would be computed and copied into this matrix. And then I print that matrix and then just to show you we can do something else here. I I saved that `correlation_matrix` as a new Excel file you can see that file. The name of that file is `correlation_matrix.xlsx`. This is my output Excel file. Correlation matrix would be copied over there. So, this is a `numeric_df.corr()` is the most important function that we have got here to compute that for me. Okay, okay, `Numeric_df.corr()`, `Numeric_df.corr()` and then I have got correlation matrix over there. End of a story. I print that and you can see the results. Okay, now what we have got here, this is the contents of my file. You can see that and this is the correlation matrix. Very important to understand an analysis that correlation between height and height one between height and weight weight 0.64, correlation is between -1 and 10.64 is a kind of reasonable because we expect taller people to be heavier somehow as well. This correlation is 0.64 and we can find some correlation between height and sex. Why? Because sex is one for female, two for male, and usually okay male has got, males are taller. So, we can see a correlation over here as well. And have a look at here random numbers and height something around 0. Actually, if you check this column, random number correlation with all variables is something around rather small number 0.1 something because random number, random numbers are not correlated with anything. Go for another one more important than height weight, weight with height. We saw that with weight with weight one, weight with sex again because we have got one and two, 0.77 and weight with weight in

British pounds. Correlation is one because weight in pound is actually a kind of linear function of weight in kilogram just multiplying by two something. So, they are highly correlated, they are linearly correlated.

They have got many things to do with each other and that's why the correlation is 1 weight with noise. Okay, correlation is a bit lower. This is the effect of that noise. I added some noise, some random numbers with weights and now the correlation is a bit lower than one and we can move ahead with other variables. But these two are the most important rows here. So, we computed the correlation and showed that here showed that here. Okay, okay, this function is very important. We call this function for `numeric_df` which is my dataframe with only numerical variables and then this is the results of calling this function core for this sample or instance of dataframe. So, core is a function developed for pandas dataframe data types and I call it for this one and this is the results.

Then this is your time to have some practice. Try to answer these questions using the resources that you have got. Try to answer these questions. This will help you to understand these lecture more clear. One why random variable has got the lowest correlation with others. Two, if we increase the noise of random numbers added to weight in British pounds, what will be the results of computing correlation between noisy weight in BP and weight in BP? Okay, we have got a noise here and this is the results of having that added to weights. If noise is stronger than that, if random numbers added to this weight be larger and change the weight more than what we can see here in this dataset, if we add stronger noise, what would be the effect on correlation between for instance this variable and this variable or this variable? Okay, what you can learn from correlation between sex and height and sex and weight. Consider that these sex is a kind of label. This is a kind of nominal categorical variable, but the values are numbers. And this way we managed to compute the correlation. And because usually ladies are lighter and shorter than man, we can see a kind of positive correlation between height, weight, and sex. Try to understand that you may change the contents of this data file.

For instance, make sex two for females and one for males and run this code once again and see what is the correlation between sex and height and weight. Now when sex is two for female and one for male. Okay this is something to do practice to try that on iris dataset. You have got the iris dataset, three classes, four features. Of course that over there when you open that iris dataset you need to remove a non-numerical variables or columns and then compute the correlation and realise the correlation between four numerical features which determine the different characteristics of Iris fluff. Okay, that's it. Just I use that runtime run all option to run that once again for you. Results won't be changed. You can do that yourself back home, you have got access to those codes and so on and that's it. Thank you very much. Take care and bye bye.

Video 2.6: Unsupervised Learning – K-means Algorithm Demo

Okay my friends, let's have a nice try in clustering. What I'm going to do here is, okay, this is our Python notebook in Google code Drive. This is `celebs2.csv` is our dataset. In that dataset again we have got three columns of height, weight, and sex of some celebrities. We are going to use the first two columns as features and cluster these data samples into a few clusters using k means algorithm. And then we can do some like changing the k or the number of clusters or changing the clustering algorithm and trying something more and learn some new items over there. So, let's get back to our Google call app. This is the drive where is the data file. Here you are. This is `celebs2` data file is csv or,

separated value file. You can see the height, weight, and gender labels for 91 data samples. And where is my Python notebook K-means tree?

K-means tree is the name of that notebook. I'm going to open that right away. You can see that here. Here you are the first cell. Okay, again importing many libraries that I need. Then trying to get connected to my Google Drive and opening this celebs data file which is in this directory in my Google Drive.

And then over here you can see that we use read csv function in pandas to read the contents of that file and put it here in this variable which is an instance or object of class pandas dataframe. And over here of course that we have got all three columns. I just want to keep two of them because this class label I don't need to have it in my that gender label class label. I don't want to have that in my clustering. Clustering should be done based on those features which are height and weight. So, over here you can see that I just keep all the rows but I just keep columns zero and one. This zero to two in Python means zero and one column, zero column one variable height variable weight okay and then the shape okay $90 * 2$ and then over here important cell. I have described this important cell for you over here in this notebook we use this function. This is a part of our scikit-learn library. You can see that here from Sklar cluster sublibrary import kmeans and over here I use this function to somehow generate a new model of class.

K means this is the name of that variable. A new instance, a new object, a new K means model. Main hyper parameters should be defined here. Main hyper parameters should be defined here. What are those main hyper parameters that when I define K means model I should declare one of them is number of clusters? How many clusters I may have over here? In this example, we start with two maximum iteration to determine the cluster centers. We start from random cluster sentence. Then we go for an algorithm to find the best cluster centers after somehow allocating the closest samples to each cluster and then to each cluster center and then finding the new cluster center. And we continue like this and the maximum number of iterations that we are going to continue this changing of cluster centers and assignment of items to cluster centers, okay, 40 times. Usually we will finish much sooner than 40. And then the algorithms, we have got different algorithms for training or determination of cluster centers. One of them is Lloyd that is selected here. You can select LCON as well. And there could be some few more algorithms established in a K-means a library of SKLR circuit layer.

Anyway, this is definition, this is a construction function which determines what are the hyperparameters of this K-means model. And then we should train that training over here using my data samples height and weight. 90 data samples height and weight are there and we call function feet for K means model. And this way after a few iterations starting from random cluster sentence, the model converts towards a kind of fixed cluster centers and fixed allocation of data samples, my 90 data samples to those clusters. And then just to show you what is the type of K-means, we know that we have got some standard data types like integer, float, etc., character strings, etc. But K-means actually is a kind of abstract data type. K-means, my variable is a model of K-means clustering algorithm. This is an object, an instance of K-mean clustering algorithm defined in cyclic layer. Okay, okay and over here when I ask Python to show me the type of K means I will get lots of information about that.

This is not just integer float or something like this. We can see many information about that data type. This variable K means which is an instance of that and then okay. What about clustering of data using that trained model? What about clustering data using that trained model? Have a look at here

K means predict function predict which is one of the psychic layer functions. We will give this data and this is the results of clustering assigned to this array. This list or array you can see that the first data sample belongs to cluster as well as many other data samples. And after that we have got some data samples assigned to cluster one. Naming indexing of clusters started from zero and because I have got just two clusters here because I just have got two clusters here you can see these are cluster labels assigned to them. Then I use in D cell I use a scatter plot. Scatter plot to see two things, one colors. Here shows the cluster assigned to those data samples. Here you can see height in this X-axis and weight over here in Y axis. And these two big pink circles, they are cluster centers. After calling the feet algorithm, after starting from random cluster sentence, here you are the final cluster sentence. And then okay, here you are again showing you the cluster labels and these are the cluster centers. Cluster center 1165 and 53, cluster center two for that yellow samples. Cluster one is 181 and 84181 and 84 here you are. This is the cluster Center for the first one, this is 165 and 53165 and 63. This is sorry 53165 and 53 which is the cluster center. The first cluster center which is here and now after running that allowing me to run all the cells. Something that you need to do in some cases is allowing that notebook to have access to your Google Drive. After that you will have the results. You can see the results over here. If we go for pure random selection of initial cluster centers, the results of clustering could be a slightly different in each run. And what about practices that you can do here one studied the effects of changing the number of clusters to 3:00 and 4:00. This parameter N cluster in K means constructive function. You can change it and see what are the results. Studied the effects of changing the cluster algorithm from Lloyd to L Contrite it and see if there is a big difference over there. Study the meaning of intercluster distance and intra cluster distance. These are two important things in clustering. What is the meaning of inter cluster distance?

What is the meaning of intercluster distance? Try to realise that then answer please. After a good clustering, the intra cluster distance to inter cluster distance ratio should be either maximum or minimum. Which one is the correct answer? Of course it depends on the definition that you managed to find for intracluster distance, inter class, intra cluster distance and inter cluster distance and then you will see this ratio. Intra to inter in a good clustering should be minimum or maximum. Then develop a function for yourself a Python function to compute the intracluster distance and intercluster distance. Input parameters to that function could be feature matrix X and cluster labels predicted by your model predicted by your model which are. Let me get rid of this one. I should run this cell. I will run all of them once again to have this removed. This is an edit mode. Edit mode of notebook in running mode. Now we will see that done more reasonably. Okay. Let's wait for a second to have the result, okay, okay, okay, okay, now I have got my comments back. Come on boy, do something for me. Okay, now I'm fine right? Okay, so develop a function to get X and cluster labels predicted by your algorithm. For instance YK means variable here and the returned value by your function could be should be intra cluster distance and inter cluster distance. And then you may try clustering of iris data set. Go for different number of clusters from 2:00 to 4:00 use Lloyd algorithm and then which one provides the smaller intra to inter cluster, intra to inter cluster distance ratio. So, this is a good practice. Go and cluster iris.csv, iris flowers dataset. Use different number of clusters two, three and four for each case. After clustering with K equal to ask your function to compute intra cluster and inter cluster distance. Compute and show the ratio of them. Then go for three and four and this way, because we know that in a successful clustering, intra to inter cluster distance should be minimum. For instance, again, we can realise if clustering with 2-3 or four was the most reasonable clustering of iris flowers data samples. That's it. Thank you very much my friends. Thank you very much for staying with me. Take care bye bye.

Video 2.7: Supervised Learning – Regression and Linear Regression Demo

Hello my friends once again. In this part we are going to check and practice linear regression algorithms. We have got this Python codes, `lrg1`, which is a linear regressor and this is our dataset. Contains 2000 data samples of different apartments and HDBs in Singapore. In Singapore, public apartments would be called HDB. Anyway, we are going to use this dataset and we have got the information and data and different features and attributes of 2000 HDBs or apartments in Singapore. What we are going to do is developing a model that would be able to guess to estimate the resale price. Our output variable in this linear regression model is resale-price based on two independent input variables of flat area in a square meter and the lease_commence_date. So, we are going to develop a linear regressor model. The output is resale-price and we are going to guess the resale price of different HDBs in Singapore based on the area and lease_commence_date. Of course that the first thing that we are going to do is showing you the dataset.

Again, you have got access to that data set. Here you are resale-sample.csv, resale-sample.csv, that's our dataset. Let's open that. You can see that we have got many variables over here. For instance, this column is months, town, flat_type, block, and the last one is resale_price. The one that we are going to estimate based on two independent input variables - lease_commence_date. You can see that lease_common_date here, which is actually the year and floor area in a square meter. You can see that here. So, we discard all other variables. Some of them are categorical, nominal, some of them are numerical. Anyhow, we will ignore all of them. We will discard all of them from this linear regressor model. We just rely on this column and this column as our input and this column as our output and see how we can move ahead with this example. Where is the Python code `lrg1`? Let me double click on that and our nice Google collab will open that for me.

Okay, okay, over here, just as ever, importing the libraries that I need. The model that I'm going to use is a `LinearRegression`, which is a part of our `sklearn` library. Again I use this function `train_test_split` to divide that dataset of 2000 samples into two individual non-overlapped subsets for training and testing of my model. And this is again getting connected to Google Drive. This is the path and file name. `resale-sample.csv` is the file name. We use `pandas read csv` to read the contents of that file and the results, the contents of that data file would be loaded into this variable `dataset` which is an instance of a `pandas dataframe`. The shape, okay, 2000 rows, 11 columns and this is the description of our dataset. `dataset.describe()` calling function `describe` in `pandas` for this dataframe and you can see the description of different variables. Of course that usually these describe only would be done for numerical variables, not categorical variables. And then in the next cell over here, I'm going to show resale price as a function of floor area in a square meter.

So, vertical axis `y` is resale price, `x` is floor area square meter. And you can see this, you can see the results in this scatter plot. Floor area square meter over here, resale price in Singapore dollar in vertical direction. And then we are going to show you another plot which is actually using `VUC` burn instance and showing you a kind of histogram of these floor area square meter. Later, if you want to practice that, you can change this variable to any variable and then this `seaborn` can show you the histogram of that variable. What is the function that we use? `distplot` is the function that we use over here. And you can see this is the histogram of our floor area in a square meter. So, this is the histogram of that. Most of the flats are around this area, something around 80,85,90 meters. The popularity, the density of the frequency of flats around 90 meter is higher than that. We have got some very small flats and some very big one, but most of the flats are around 90 square meters and

so on. Now after a kind of visualisation using distplot function of seaborn calling that for this particular column of my dataset, data frame, and as I mentioned, you can put whatever you like here. We are going to prepare our data set, then split that data set into two parts for training and testing.

And moving ahead, what we are going to do here is firstly we just focus on floor_area_sqm and we are going to, yes, the resale_price as our output variable as a function of that. And later we may go for another dataset where we have got both floor_area_sqm and least common state as our two variables and building or guessing our resale_price based on those two variable. But in this cell, the input variable is just floor_area_sqm and resale_price is just a function of floor_area_sqm. And then we are going to divide our data set into two non-overlapped parts for training and testing. This train test split function of sklearn will do that for us. x is our features, y is the class labels or outputs. Over here this is the regression. So, the output y is the resale_price and the test size is 0.2, means that 20% of data would be considered for testing, 80% would be allocated for training. And then we develop a new example of LinearRegression models called regressor calling LinearRegression function inside sklearn.

This constructive function build up a new regressor model for me in this variable regressor. And then we train that using function fit with our training features and variables. Only one independent variable or feature is over there and then we would like to have those coefficients. And coefficients are the coefficients of the linear model that we managed to develop based on our data and these are the coefficient of that linear model. These print functions actually print that my model called regressor.intercept. This regressor is the name of my LinearRegression model, the name that I selected for this variable or instance of that class, object of that class and this regressor coefficient over here, just one number, is the other coefficients rather than intercept over there and then we go for a prediction. Okay, after training this model using test samples and predict function, fit function for training, using training features and training outputs, predict function for prediction and testing using these testing features, testing inputs and answer are here. And then I can print the predicted values by my model and real prices that my dataset has got for my test samples. And you can see that for instance, in this case, my first data sample for testing, of course actual price is 530,000. Price estimated by my model is 543,000 or so. This example test sample, the real price is 363,000. My model estimated that as 364000 and then we can plot that. You can see the plot. Actually the blue color shows the actual price of my test samples and orange shows the price predicted for that. Of course that I don't plot that for my old data samples but just for 25 data samples the first 25 data samples that I have got. So, I copied the first 25 data samples of DF my data frame here and then I just showed the real and actual and predicted price of that actual and predicted price that I computed them here and inserted these to my dataframe, dataframe that I use here to see the results. Actually here this df is my variable. This is a new dataframe variable. From that dataframe, I just copied these two variable y_test and y_pred and put them over there as two columns of this new dataframe called df.

And then I use that dataframe and show the first 25 data samples of that. And this is another scatter plot of test samples over here. This is the area, this is the estimated price. And this is my model linear model fit between these test data sample. And you can see that over here, the grey dots actually are the scatter plots of my test data sample. And this red line is the linear model fitted over there and the model that I used to estimate the price just based on floor area, square meter. And then I can print the mean absolute error between test and prediction, mean square error between test real prices and predicted prices. And we can show the S square root of mean square error as

well. The S square root of mean square error as well. So, I have got mean absolute error MAE, mean S square error, MSE and S square root of mean_square_error. And these are the results of my model 70,000 is the mean absolute error in average between the real prices, actual prices, and prices predicted by my model just based on the area area in a square meter.

If we want to add the next variable, that would be very easy, but plotting won't be that easy too, because I need to go for three-dimensional scatter plot or something like this. And model instead of a line would be a plane, a hyperplane, a hyperplane, linear hyperplane. I haven't gone for that and now for testing our model, this red line is my linear model, would be able to guess the price of an HTTP in Singapore just based on its area and of course that won't be 100% accurate. For instance, according to my model, if the area of a flat in Singapore is 255.5 meter, then the price predicted for that would be 1,027,000 something, 1,027,470 sing dollar. And you can see this number, this area over here and see the estimated price of your flat, your HTTP. For instance, if I go for 125 square meter and run the model, the estimated price is 553,000. Now it's time for your practice. You can use the celebs to dataset. Go for that, use the celebs to dataset. Try to do something over there. We would like to develop a model, a linear regressor model to guess the celebrity's height based on their weight. Means that the input variable is weight.

The output variable we are going to estimate that is height. Height is F of weight, try linear regression. Just keep the first and second variable because the third variable is the agenda, we don't need that. So, just first and second variable. First one is height is my output variable, second one is weight is my input variable. Use 80% of data samples for training, 20% for testing. Report the MAE, mean_absolute_error and mean_square_error of your model. This won't be very precise. However, this is a try to guess the height of someone based on its weight. Thank you very much. That's it, and this was our linear regression example.

My friends, you may ask what if we wanna have that price of HDB's resale price of HDBs estimated based on two independent variables.

Okay, the code that you can see here, the notebook that you can see here, lrg1p does something for us like this. Again, we will have the same dataset, but after that descriptive etc., dataset.describe() opening the data file etc., we are going to build up a dataset, a feature set of these two variables or columns of our data file. So, floor_area_sqm is my first variable and the lease_commence_date over here is my second variable, and they are my independent input variable. And now resale_price in this notebook, lrg1p, would be estimated based on these two. So, over here you can see that I have got two columns in this input variables or input features matrix, and output is the resale_price. And then we develop our linear regression model called regressor. Before that, we split the training and testing datasets into two parts, 80% for training, 20% for testing, and then after training our linear model using this fit function called for this regressor object or instance of class LinearRegression.

We can see the coefficients over here, the intercept_ and then other coefficients of this model. Over here because resale_price is a function of two input variables plus intercept. Actually, we have got three coefficients. One multiplied by floor_area_sqm, another one multiplied by lease_commence_date, and another one would be added to them which is the intercept_ and then this would be the resale price estimated by our model. So, this is the intercept and these are two coefficients multiplied by floor_area_sqm and lease_commence_date. Okay, this way, let's move ahead and see what we have got as our result.. prediction here regressor.predict() with our test samples and here you are, the estimated price and Mean Absolute Error, Mean Squared Error, and Root Mean Squared Error. You can see them over here. So, now in this example lrg1p, we have got

two input variables and resale price would be estimated based on that and this is our new Mean Absolute Error and Mean Squared Error and again, to test that for a HDB apartment with 120 meters.. 123 meters area built in 1989, lease_commence_date is 1989. What would be the estimated price by my model? You can see that here \$536,587, approximately \$536,000 is the price estimated by this model. Thank you very much.

Video 2.8: Supervised Learning – Classification and KNN Algorithm Demo

Okay my friends, let's see how we can try a K-Nearest Neighborhood classifier. The scenario is, I have got this iris dataset. This iris dataset contains 150 data samples of iris flowers, four features extracted per data sample which are petal lengths, petal width, sepal lengths, sepal width, and we have got three classes in this iris dataset, one of the standard data sets in machine learning. What are those three different types of iris flowers that we can see in this 150 data samples dataset? Iris Setosa, Versicolor and Virginica. The classifier that we are going to use is a K-Nearest Neighborhood one and the code can be found in knn1.ipynb. We will try different k's and we will see what else we can do with this example. So, let's get back to our notebook. K-Nearest Neighborhood example name is knn1. I am going to run that, one difference between this example and other example, of course that in our drive we have got a copy of that Iris dataset. However, to show you that we have got some other options and alternatives to read a data file, this time I'm going to read that iris dataset from this URL and to do that, okay, I know the name of those variables. First column is sepal-lengths, then this, then this, then this and the last column is Class label, and this is the URL that I can use to load iris data from that. And then, I use this read_csv with this URL and this is a list of name for variables. This Iris data hasn't got any variable name, any header row, but using this list now I can have them. So, read_csv and the result is an instance of pandas DataFrame dataset. I can show you the first five rows of that.

You can see that this is sepal-length, sepal-width and other numerical features extracted for those data samples, and this is class. My first five data samples belong to Iris-setosa and this is the shape-150 rows, five columns and type of that dataset. Again, because this is a kind of abstract dataset, you can see different information metadata available in this cell. When I say show me the type of that dataset, you can see lots of metadata regarding the pandas DataFrame, abstract datatype and in particular this variable and instance of pandas DataFrame called dataset. Then I will separate the first four columns, all rows; first four columns means all the columns but not the last one and assign that to X. Actually my dataset, my training features and the last column-- column number 4 is the labels that Setosa, Versicolor, etc., assigned to y. So, now X is my feature vector with those numerical features of different data samples, y is my class label. Now, in the next cell we are going to configure our model which is a K-Nearest Neighborhood class where before that, over here, I just split my X and y, my dataset into 80% for training and 20% for testing.

Using this train_test_split, this variable this hyperparameter here means that we are going to randomly separate 80% of data samples for training and 20% for testing. However, this would be replicable, means that when we use this random_state=1. Of course, that the separation or splitting of data samples into training and testing would be random. But, anytime I run this example the same 80% would be selected the first time random order, the next time we will have the same sequence of random numbers. But if I change this random_state to something like None, okay, that would be a pure random assignment of my samples into training and testing subset. And every time that I run my model, I will have different samples as my training samples and different samples as my testing

samples. Therefore, usually because in during the education and testing of models we would like to have our results replicable, we don't go for `random_state = None`, but we will go for a number here. That number actually is the seed of Python random number generator.

Next, after splitting my dataset into training and testing subsets, I'm going to develop my model. And to do that, to have the best results, a kind of normalisation isn't bad. Why this is my test subset? Test subset before normalisation. You can see that this is the first feature. The scale is 6-7 something the next 1,3,2,4,2,2,4 something next one between 4 and 7. The last one is just below 2. Of course that this is not a big deal, but it's not bad to normalise our data. And in this cell you can see the normalisation of data using standard scalar which transform our data to a kind of Gaussian normal distribution with average equal zero, mean equal 0 and standard deviation equal 1. So, standard scalar is one of the possibilities in normalisation in `psycic learn` in Python. And there are different approaches in normalisation. If we go for a standard scalar for each column for each variable like these, for instance, simple lengths, we will normalise that the way that the average of this variable, this column would be zero and the standard deviation would be one. We just compute the average of that.

We will subtract that average from each data each number here and then we divide that by the standard deviation, results would be a sepal-length normalised. Average is 0, standard deviation is 1 as a standard Gaussian distribution. Same would be done for all three other features that we have got here. Results of normalisation. This is my dataset before normalisation. This is my dataset after normalisation. Again, this is my first variable, This is my second variable. But now the average of this column first column is 0. Standard deviation is one. Sometimes, and I should say in many cases in data analytics, in machine learning, normalisation is useful. In particular, when a scale of variables in different columns are different and are very different. Now it's time to develop our K nearest neighborhood model. We import KNN `KNeighborClassifier` model from `psycic learn` and then this is the name of my KNN Classifier, an instance of K Neighbor Classifier and the only hyper parameter that I should set when I construct this model.

When I call this function, I actually ask that to construct an instance of `KNeighborsClassifier` or model and that model is here Classifier and the only hyperparameter that I should provide is number of Neighbors or K `KNeighborsClassifier`. What is the K selected 19? I have opted for 19 so K or number of Neighbors is 19 and now I have got a new `KNeighborsClassifier` model here. Then I train it using `fit` function using `fit` function and my training data sample and training features and training labels. So, now my Classifier has been trained. Of course, the training for `KNearestNeighborhood` is not a big deal. And then I have to test it using the function `predict` and my `X_test`. The features here you are the labels assigned to the class labels assigned to that test samples. And then we have to see the performance evaluate the performance of our model in this classification. Two useful functions available in `psycic LAN` library. One of them is `confusion_matrix` and other one is `classification report`. I call both of them parameters that I passed to these two functions. Is `whitest` the real class labels of my test sample and `y_pred` the predicted class labels for all of my test samples. And here you are the `confusion_matrix`. In a successful classification, the numbers in this diagonal of this `confusion_matrix` is maximum and the other numbers should be 0. Over here we can see that one of the numbers out of that diagonal is 2. Means that my performance, my accuracy, my precision, or recall won't be 100%, won't be complete.

How we can interpret this `confusion_matrix`? We have got three classes. Over here, these 11 shows that we had got 11 samples of the first class all correctly classified as the class one or Setus.

Numbers here suggest that out of my 13 actual verticolor samples in my test set, I classified correctly 11 of them as verticolor, but wrongly I classified two of them. I detect two of them as virginica out of 6 virginica that I had got in my test sample, all classified correctly as virginica. And then this is my second results results of calling that classification report function. And you can see that for Iris Setosa vertical virginica precision recall, F1-score support means number of samples on those classes can be seen here. Average accuracy is 0.93 and the only problem is with recall of vertical which is 0.85 because a 2 out of 13 vertical classified as non-vertical. So, the result is not bad. Out of 30 test samples, I classified 28 of them correctly but two wrong. Question is can I make it better? Okay, the first thing that I can do and you can see that in this cell is going for different case. In this loop I try K from 1 to 39. I construct a Classifier with this number of K assigned to variable I in this loop and I would be changed between 1 and 39. So, I construct Classifier with these K or `n_neighbors = i` and then I train it with my training data sample. I test it with my test data sample and then I will show you the results or the number of incorrectly wrongly classified samples. Something like this happened when predicted label won't be equal to the real class label and we will repeat that for all is between 1 and 40. So, we construct the Classifier, we train that. We ask it to predict the class labels. Then we count the number of wrongly classified samples and then we plot that. Here you can see that the x-axis shows the different K values between 1 and 39 and Y value is the error number of wrongly classified samples and whenever these error number of wrongly classified samples is 0 means that my Classifier for this particular K has got 100% accuracy and no problem.

The first one that I tried was with a number of classes equal 19. You can see that here. And in this graph it is obvious that for K = 19 the error is 2 out of 30 or 0.65, 0.065 something like this 2 out of 30 something around Okay $2 / 30$ is 0.66 something like this. And this is 0.66, 0.0666 which is my air but this curve also shows that for instance with K = 6 or 8 or 9 classification accuracy would be 100%. And for K=3 as well. Now I develop another model non `n_neighbors` or K is 3i fit it, train it, and then I ask it to predict the test samples, assign class to test samples and now and then I print confusion matrix and classification report and you can see that with K=3, now every numbers out of this diagonal is means that there is no wrongly classified sample. And you can see that all the numbers, precision, recall, F1-score in macro average, weighted average or per class, all are one means 100% correct classification things to do for you.

Over here, Try KNN with three, five, and seven on your iris dataset without any normalisation. So, the things that you need to do now as a practice, I advise you to do that. Ignore normalisation, ignore normalisation. And don't forget that when we normalise something, we use the parameters of training subset and we apply parameters of training subset to normalise training samples and testing samples to get. So, training sample, testing sample would be normalised using the parameters of training subset. And now if we ignore that normalisation part, what would be the classification accuracy and the performance of our model? You will see if normalisation is useful here or not. Is it more accurate or less? Then you have got another dataset celebs2.csv. Use that, separate these with 70% for training and 30% for testing. Develop a Classifier that would be able a KNN Classifier. A KNN Classifier that would be able to classify celebrities of that data file into two gender classes, one for female, two for male. Based on which features, classify celebrities into two gender classes, female and male based on what height and weight. These are our feature. So, try to find a way to load that celebs2.csv file into a pandas dataframe. Then separate the first 2 columns as your features, use the last column as your class labels and then develop a KNN Classifier for that. You can go for normalisation or do that without normalisation. Finally, try different case 1,5, and 7, and

realise how it works in classification of celebrities into two gender classes. Thank you very very much my friends. Good luck.

Video 2.9: ANN as a Classifier Demo

Hello everybody. Let's try to see how we can use an artificial neural network in a supervised learning activity as a classifier. This is the definition of that supervised learning task or classification task. We are going to use an artificial neural network to recognise diabetes. This is our dataset of some lady visitors of a clinic. Some of them are suffering from diabetes, some of them are normal. We have got 768 samples and eight features such as pregnancy, glucoses, BMI, and so on. The last column of that dataset is actually the outcome 0 for normal people and 1 for people who suffer from diabetes. And this is the code ANN110.ipynb that we are going to run and see how we can design a neural networks as a classifier, which is actually a kind of supervised layer.

Okay, where is my dataset and notebooks? Okay, here you are. The dataset, this nh over here in the name of this dataset means no header. Similar dataset here, but it has got the header. I'm going to open that to show you the variables, the features, the columns. Abort we use that in our program. Similar, but no header for this one header for this one. Let me open that. Okay, this is my dataset. You can see the class label, 1 for diabetes positive, 0 for normal. This is the outcome. Other variables, the number of pregnancies, glucose, blood pressure, and so on. Doctors consider those features as features who have got something to do with the detection of diabetes. So, let's get back to the quote ANN110. Let's run that because in the practice you are going to change the runtime type. I'm going to just run this example and see what's going on.

To implement our neural network, I use tensor flow and Keras. They are popular. Tensor flow and its subclass Keras are very popular amongst neural network developers. Again, opening that datasets 228 nh.csv file here. But instead of using pandas and opening that as a data frame, I use another function loadtxt. And using loadtxt, I just open this file and instead of using esplit functions in psychic layer, I'm going to somehow, I'm going to somehow use let me get rid of this one and ask system to remove that somehow. Let's let's do that and get rid of that. After that, okay, cancel, no problem. I don't know where I can put this. Okay, Okay, I'm going to continue. Okay, we use this loadtxt. We use this loadtxt and then over here we read the contents of that file and put it here. Then we manually separate the first 500 data samples for training and column zero to seven which are our features saved here in xtr. For ytr, actually the same data samples but the last column which contains those 0-1 class labels, the rest between 501 and 768. Around 268 data samples would be used for testing. Again, the first 8 columns as zero to seven actually as my features. The last column that outcome is the class labels. And now I'm going to develop my neural network. Sequential is the name used by tensor flow to show that this model, the name that I select for my neural network is model is a kind of feed forward neural network. This is the first layer input dimension.

Okay, Eight neurons in my input layer because I have got eight features, eight features for that the diabetes suspected to suffer from diabetes people. So, 8 means eight features here. Activation function of the first hidden layer is relu. So, activation function of the first hidden layer is relu. 20 neurons in that hidden layer and eight neurons in the first input layer. Next hidden layer again here activation value 20 neurons and dense over there means a fully connected layer. We have got different types of layers in Tensorflow from for instance we can go for convolutional etcetera. But dense means fully connected perceptron like neurons in this layer. The last hidden layer again 20

neurons, a dense fully connected one activation relu. So, one input layer with eight neurons, two hidden layers each with 20 neurons and one output layer. Activation is sigmoid because sigmoid easily can move from zero to one and we know that the class labels here are zero and one and this output dense layer just has got one nerve just has got one nerve. So, output layer is a dense one, activation function sigmoid good for classification but just one neuron over there because the output of that neuron is either zero or one which shows the class.

Now we use the function compile in tensorflow to configure to construct our neural network. The name of that neural network is model loss function. Error function is binary cross entropy, very good for classification. Training algorithm or optimiser is Adam. We have got different training algorithms here. One of them is Adam stands for adaptive momentum and the metric to evaluate the performance of classware is accuracy. Actually accuracy suggests how many data samples have been classified correctly divided by total number of data samples. These would be the accuracy. So, three hyper parameters defined here. What is my loss function binary_crossentropy. What is the training algorithm or optimiser adam and what is the evaluation metric which is accuracy. Then we train over neural network using training features, training labels, calling fit, or training function for these neural network called model. How many training epochs? 300 batch size to apply the weight modification over finishing a complete batch of input training data samples is 10. So, important point is these epochs or the number of training iterations. Then we evaluate our model after finishing the training. We evaluate our model using these test features and test labels. Model function evaluate output accuracy and then we print that accuracy results of training.

You can see that results of training over here. We start with rather low accuracy and low accuracy and high loss. Over here you can see that when I start accuracy is 0.42, training accuracy and loss is 3.94. Then after 100 training epochs actually accuracy reaches to around 77-76% and loss is 0.48. I continue training up to the last training epoch which is 300 and then this is accuracy using test sample. Using test sample, this is accuracy over the training sample. This is the results of calling that function evaluation and this is the results of the accuracy of test samples. Then we would like to have the confusion matrix and classification report. So, I call this function predict for my neural network model using these training features, sorry these testing features. So, function predict dataset is features for testing xtst neural network name is model. And this is the result of using that model now trained for 200 epochs to assign classes to those test samples. And we have got the real classes of those test samples. Here you are white tst is an array which comprises the labels and classes of those test samples. Features of those test samples are here. So, and now we can compare the predictions with real classes of those test samples.

And we know that zero means that this visitor is normal, one is that this visitor suffers from diabetes. And then we call confusion matrix, Confusion matrix and classification report. We call confusion matrix and classification report. So, important functions over here, one over here in this cell. I reviewed that once again, model compile, then model feed, then calling evaluate with testing data, testing subset, then calling predict to have the class labels assigned to test samples, saving that on Y_pred variable. Then comparison of ytsd and y_pred using two functions available in psychic line, confusion matrix and classification report. This is my confusion matrix. 149 zero labeled as 0 means normal detected as normal 32 normal classified as diabetes 37 diabetes recognised as normal 49 diabetes recognised as diabetes and you can see the precision and recall and F1 score for class 0 and Class 1. Actually we had got 181 normal people, 86 people suffers from diabetes in the set of 267 test samples, accuracy 74 and micro averaging and weighted average of accuracy is something that you can see. So, we configured our neural network classifier here.

One input layer with eight neurons, three hidden layers each with 20 neurons. Those number can be changed. You can remove one of the layers. You can add another hidden layer. You can change the number of neurons in hidden layers. Don't touch the number of neurons in input layer and output layer but number of neurons in hidden layer it's up to you. You even can change the activation function of hidden layers but don't touch this sigmoid activation function of output layer. Instead of value you can go for linear you can go for sigmoid again as activation function of your hidden live. But know about the things that you can do change the runtime type. Okay, I'm going to show you here. Have a look at this runtime type and I ran it with 800, A 100 GPU and high RAM. You can switch it back to CPU and run it once again and see the difference. Differences won't be in accuracy but in the speed of training and evaluation of your model. So, we can go for CPU, A 100, GPU for instance and try and see the differences.

Change the activation functions of the first three layers to linear. First three layers means that those three hidden layers to linear and then see the results. Try to run the program from the scratch and see the results. Bring back the activation functions to real. You once again change the training epochs step respectively to five with five training epochs. Try to run your model and see the results. Change your training epochs to 5000 and see what will be the difference over there. So, normally the number of training epochs in my example is 300. Change it to five, see the results. Change it to 5000, see the results, you need to run it from the beginning and then final things that you can do, try to normalise your data. For instance, using that standard scalar normalisation that you saw in previous examples, try to normalise your data. Both training and testing subsets should be normalised after that. After that normalisation, see if a performance is better, accuracy is better or not. Thank you very much my friends, have a good time ahead. Good luck and bye bye.

Video 2.10: Advantages and Disadvantages of ANNs

Now let's talk about advantages and disadvantages of artificial neural networks. If you want to list the most important advantages of artificial neural networks, we can go for ability to learn complex patterns. ANNs can model and learn non-linear and they are good in learning non-linear relations and in modelling non-linear relations and in going for non-linear classification. So, any complicated non-linear relation between input and output can be modelled by an artificial neural network. They are highly adaptable. Their adaptability is very high, they adjust to new data and improve their performance with more learning. Of course, then they need to be properly designed, but basically a neural network would be able to for a kind of retraining or fine-tuning, fine adjusting of its parameters, and it can adopt even without add new training or fine-tuning to handle some similar new samples.

Generalisation is another advantage of our neural network. Once trained, ANNs can make predictions or classifications on unseen data successfully and rather accurately. ANNs are capable in doing parallel processing. Neural networks can handle multiple computations simultaneously. Therefore they are computationally effective. Wide applicability is another advantage of our artificial neural network used in various domains like image and speech recognition, natural language processing and financial forecasting, all successfully, all with high performance shows that the range of applications for artificial neural networks is rather very high and they are fault tolerant. ANNs performance is not significantly decreased by minor errors or noise in our input data, in particular this feature. This advantage is important when you are dealing with semi-structured or unstructured data, so they are fault tolerant as well.

However, if we want to list the disadvantages of over neural networks, they are on not very small models to be implemented and we need good computers. We need good hardware to implement them. They are not very simple models, so they are computationally heavy. High computational requirements is a fact when we use actual neural networks. Again, their nature is black box. They are not very explainable. Of course that we have done a lot to make them more explainable. However, the nature of neural networks either shallow models or deep models, they are a kind of black box models and not very transparent and explain overfitting risk always is there in machine learning. Overfitting is a real risk and it has got something to do with the number of parameters that you have got inside your model. The higher the number of parameters, the higher the risk of overfitting. And inside neural networks we have got many parameters. So, overfitting is a risk that we should consider when we deal with neural networks, when we train them and so on.

And of course they are data thirsty dependency on large datasets. The larger your model, the larger your neural networks, the higher this data requirement. Usually we need big data. Usually we need big dataset to train our neural networks. And we can consider neural networks as a kind of data thirsty model. They have got many hyperparameters. When you use a neural networks there are many hyperparameters to set and again the behaviour of them depends fully on the setting of internal parameters. So, setting of hyperparameters is something that you should be careful about and proper training is important because setting of internal parameters is important as well. Without those two settings, proper settings of hyperparameters by you as designer, as user and proper setting of internal parameters during the training, the performance of your neural networks can't be high. And in some cases, those training or proper setting of internal parameters, weights, biases etcetera can be considered as a kind of convergence issue. We need to go for a kind of proper training. We should avoid overfitting, but we should converge towards a low enough, adequate enough performance in overtraining and adequate enough and low enough error in our training after proper training.

So, convergence is another problem. We should train them properly and we should come across a low enough error after proper training. This is the meaning of convergence and in some cases, in some cases, because the structure is not fit with the problem, because of lack of data, low quantity in your data or low quality in your data, we may face a kind of unsuccessful training of our neural networks. So, we saw the advantages and disadvantages of our artificial neural networks. However, however, it should be mentioned that whatsoever they are the best machine learning models that we have got, in particular for processing of complicated data and in challenging AI problems.

Video 2.11: Conclusion

My dear friends, we have started with definition of Machine Learning and three different approaches in machine learning: supervised, reinforcement, and unsupervised learning. We saw some applications of and some examples of unsupervised learning, for instance, clustering, K-means clustering. Also, we saw how to go for regression and function estimation and classification. And we ended that module with a review of artificial neural networks. Nowadays, machine learning techniques are being widely used to solve different challenges in the field of AI. Real-world applications of machine learning, they have been proven addressable by modern machine learning techniques.

We have got many challenges in many cases, in businesses, in different applications, we need classification, we need function estimation, we need many other things that modern machine learning can do for us. Again, supervised machine learning, unsupervised machine learning, reinforcement learning; they have got their own applications. We can't say that which one is more important. We can say that we should go for a proper approach considering the problem, the data, our goals and motivations, and something like this. So to sum up, again, machine learning is very important. Why? Because it has shown that, it has proven that it can help us with solving many challenges in the real-world applications. And again, that's up to us to have good data, enough data, and to find a good model, unsupervised, supervised, reinforcement learning, whatever to address the problem in hand. This way machine learning would be successful.